

Contents

Observability Plan — Tiered Comms Resources	1
Purpose	1
Resources to instrument	1
Delivery shape	2
Do not instrument (yet)	2

Observability Plan — Tiered Comms Resources

Purpose

When a user reports “I didn’t get the notification” or “the bracket didn’t update live,” we need to pinpoint which comms tier failed. The re-tiering introduced three critical resources, none of which currently emit metrics. This doc lists what to instrument and why, so a future Observability sprint can pick it up without re-deriving the gaps.

Not urgent. Add to the list; do not let it block feature work.

Resources to instrument

1. SSE broker connections

File: backend/src/lib/sseBroker.js, backend/src/routes/events.js

What it is: Long-lived HTTP streams, one per logged-in tab. Tracked in an in-process Map keyed by topic and subscription id. All Tier 2 fanout (non-game, non-push events) flows through here.

Risk: Leaked connections → FD + memory growth. No visibility on peak concurrent, per-topic fanout shape, or how long clients stay subscribed. A silently-dead write path would show up as “UIs stopped updating” with no server-side signal.

Instrument: - **gauge** — open connections (labeled by topic) - **histogram** — connection age at close (detects flapping / premature disconnects) - **counter** — write failures (res.write returned false, EPIPE, etc.)

2. Redis stream events:tier2:stream

File: backend/src/lib/eventStream.js

What it is: Single shared stream every backend process XREADs for Tier 2 fanout. Producers call publish(); the SSE broker consumes and forwards to local SSE clients.

Risk: Producer outpaces consumer → stream length grows unbounded (Redis memory). Consumer lag → stale UI across the fleet. If the XREAD loop throws and the restart logic has a bug, fanout dies silently — no client-side error.

Instrument: - **gauge** — stream length (XLEN events:tier2:stream) - **gauge** — per-consumer lag (now – last-delivered-entry timestamp) - **counter** — XADD rate, segmented by event type - **counter** — XREAD loop restarts (should be ~0 in steady state)

3. Web Push subscription pipe

File: backend/src/lib/pushService.js + PushSubscription DB table

What it is: Each subscription is a per-device opaque URL to a browser push gateway (Apple/Google/Mozilla). The backend POSTs VAPID-signed payloads to these URLs. De-registered devices return 410 Gone forever.

Risk: Silent delivery failures are invisible — the client never knows a push was intended for it, so “user didn’t get notified” has no signal. Stale subscriptions pile up indefinitely if not pruned, bloating the fanout loop.

Instrument: - **counter** — delivery attempts / 2xx / 4xx / 410 / 5xx, segmented by endpoint host (apple / google / mozilla) - **gauge** — active subscriptions (total + per-user distribution) - **job** — prune subscriptions after N consecutive 410 responses

4. (Optional) Presence store

File: backend/src/lib/presenceStore.js, backend/src/routes/presence.js

What it is: In-memory (or Redis-backed, confirm before instrumenting) map of user → last-seen timestamp, fed by the landing useHeartbeat hook.

Risk: If in-memory per-process, it doesn't survive a rolling deploy — acceptable, but worth confirming. A stuck cleanup (timer dies, entries never expire) grows forever.

Instrument: - **gauge** — entries in presence store - **histogram** — entry age at eviction

5. Training worker queue (A3b.5 — shipped)

Files: backend/src/queue/queueMetrics.js, backend/src/queue/workerResourceSampler.js, backend/src/queue/trainingHealthMonitor.js, admin endpoint GET /api/v1/admin/health/training.

What it is: BullMQ-backed training queue produced by the backend (enqueueTrainingStart in trainingQueue.js) and consumed by the xo-training-* worker app. The cutover landed in A3b.2b behind the ml.useWorker SystemConfig flag.

Risk: A stalled worker pool is silent — training:start jobs pile up in BullMQ's waiting set with no client-visible signal until a user notices their training never started. The attempts:3 retry policy (A3b.4) drops permanently-failed jobs into BullMQ's failed index, which is also invisible without explicit instrumentation.

Instrumented (shipped): - **gauge** — queue depth (waiting / active / delayed / failed / completed / prioritized) via queueMetrics.getTrainingQueueMetrics(). - **gauge** — oldestWaitingAgeMs (now - head's timestamp). Powers the "queue is stalled" alert. - **gauge** — per-worker CPU% + RSS/heap MB + active-session count, sampled inside each worker process every 30s and written to Redis under training:worker:metrics:<hostname>:<pid> with a 120s TTL. A dead worker's key auto-evicts within one missed heartbeat. - **edge-triggered alert** — queueStalled fires when waiting >= 1 AND oldestWaitingAgeMs >= 5 min. Both conditions must hold so a fresh burst that drains quickly doesn't page anyone. - **edge-triggered alert** — deadLetter fires when failed >= 1 (a permanent failure after all retries). - Both alerts dispatch via the existing notificationBus to admins; cleared transitions also dispatch.

Surface: GET /api/v1/admin/health/training returns the latest snapshot (queue counts, per-worker samples, alert flags, thresholds). The admin Health page can render the gauge inline — Bull-Board is deferred unless a real triage need surfaces.

Tuning: thresholds default to queueWaitingThreshold=1 and queueAgeThresholdMs=5min. Pass overrides through startTrainingHealthMonitor({ thresholds }) if a noisier environment needs a higher bar; the per-tick evaluateTrainingHealth takes them as well so an admin endpoint could later expose runtime knobs.

Delivery shape

No preference yet between Prometheus, OTel, or a simple admin JSON endpoint. Whatever the first consumer wants (Grafana dashboard, admin UI widget, alerting pipeline) should drive that choice. For a first pass, a single GET /api/admin/observability that returns current gauge values as JSON is enough to unblock dashboards without committing to a stack.

Do not instrument (yet)

- Socket.io Tier 1 — already covered by doc/archive/Socket_Instrumentation_Plan.md plus the in-product admin realtime panel.
- Per-request latency — generic app metric, not a tiered-comms concern.